

Q1. Write down the translation scheme to generate code for assignment statement.

List the scheme for generating Three Address Code (TAC) for the assignment statement

$x = (a + b) * (c + d)$

Answer

(i) Translation Scheme

Grammar

$S \rightarrow id = E$

$E \rightarrow E1 + E2$

$E \rightarrow E1 * E2$

$E \rightarrow (E1)$

$E \rightarrow id$

Semantic Rules

$S \rightarrow id = E$

{ emit(id.place = E.place); }

$E \rightarrow E1 + E2$

{

E.place = newtemp();

emit(E.place = E1.place + E2.place);

}

$E \rightarrow E1 * E2$

{

```
E.place = newtemp();  
  
emit(E.place = E1.place * E2.place);  
  
}
```

$E \rightarrow (E1)$

```
{ E.place = E1.place; }
```

$E \rightarrow id$

```
{ E.place = id.lexeme; }
```

(ii) Three Address Code (TAC)

Expression:

$x = (a + b) * (c + d)$

Generated TAC:

$t1 = a + b$

$t2 = c + d$

$t3 = t1 * t2$

$x = t3$

Q2. Consider the following Boolean expression:

$(a > b) \ \&\& \ (c < d) \ || \ (e < f)$

Grammar:

$B \rightarrow B1 \ || \ B2$

$B \rightarrow B1 \ \&\& \ B2$

$B \rightarrow ! B1$

$B \rightarrow E1 \text{ relop } E2$

$B \rightarrow \text{true}$

$B \rightarrow \text{false}$

(i) Explain how operator precedence and associativity affect the evaluation of the given Boolean expression.

Answer

The precedence of Boolean operators is:

Highest : !

&&

Lowest : ||

Therefore the expression is interpreted as:

$((a > b) \&\& (c < d)) || (e < f)$

Evaluation using short-circuit rules:

1. Evaluate $(a > b)$.
2. If false, evaluate $(e < f)$.
3. If true, evaluate $(c < d)$.
4. If $(c < d)$ is true, the whole expression becomes true.
5. Otherwise evaluate $(e < f)$.

Thus, **&& has higher precedence than ||**, and both operators are **left associative**.

(ii) Construct the Three Address Code (TAC) using short-circuit evaluation.

Answer

if a > b goto L1

goto L3

L1:

if c < d goto Ltrue

goto L3

L3:

if e < f goto Ltrue

goto Lfalse

Ltrue:

...

goto Lend

Lfalse:

...

Lend:

(iii) Represent the control flow using labels and conditional jumps.

Answer

a > b ?

/ \

T F

| |

c<d e<f

/ \ / \

T F T F

| | | |

TRUE-----|---|---FALSE

Q3. Consider the following assignment statements involving pointers

x = *y;

a = &x;

(i) Construct the Three Address Code (TAC). Clearly represent

(a) Pointer dereferencing

Answer

t1 = *y

x = t1

(b) Address-of operator

Answer

t2 = &x

a = t2

(ii) Represent the intermediate code

(a) Quadruples

Op	Arg1	Arg2	Result
----	------	------	--------

*	y	-	t1
---	---	---	----

=	t1	-	x
---	----	---	---

&	x	-	t2
---	---	---	----

=	t2	-	a
---	----	---	---

(b) Triples

Index	Op	Arg1	Arg2
-------	----	------	------

0	*	y	-
---	---	---	---

1	=	(0)	x
---	---	-----	---

2	&	x	-
---	---	---	---

3	=	(2)	a
---	---	-----	---

(c) Indirect Triples

Pointer Table

Pointer	Triple
---------	--------

P0	0
----	---

P1	1
----	---

P2	2
----	---

P3	3
----	---

Triples remain unchanged.

Q4 (i). Consider the following program fragment

```
while (A < C && B > D) do
```

```
    if (A == 1) then
```

```
        C = C + 1
```

```
    else
```

```
        while (A <= D) do
```

```
            A = A + B
```

Generate the Three Address Code (TAC).

Answer

L1:

if A < C goto L2

goto Lend

L2:

if B > D goto L3

goto Lend

L3:

if A == 1 goto L4

goto L5

L4:

t1 = C + 1

C = t1

goto L1

L5:

if $A \leq D$ goto L6

goto L1

L6:

$t2 = A + B$

$A = t2$

goto L5

Lend:

This TAC clearly shows:

- Conditional jumps
- Unconditional jumps
- Labels
- Boolean expression evaluation

Q4 (ii). Consider the following C program

main()

{

int i;

int a[10];

i = 1;

```
while(i <= 10)

{

    a[i] = 0;

    i = i + 1;

}

}
```

Translate into Three Address Code (TAC).

Answer

i = 1

L1:

if i <= 10 goto L2

goto Lend

L2:

a[i] = 0

t1 = i + 1

i = t1

goto L1

Lend:

Representation of array element

a[i]

or

$$t2 = i \times 4$$

$$a[t2] = 0$$

where **4 bytes** is the size of an integer.

Q5. Consider the Boolean expression

$$(a < b) \parallel (c < d) \&\& (e < f)$$

Grammar

$$B \rightarrow B1 \parallel B2$$

$$B \rightarrow B1 \&\& B2$$

$$B \rightarrow ! B1$$

$$B \rightarrow E1 \text{ relop } E2$$

$$B \rightarrow \text{true}$$

$$B \rightarrow \text{false}$$

(i) Write the Syntax Directed Definition (SDD).

Answer

OR

$$B \rightarrow B1 \parallel M B2$$

backpatch(B1.falselist, M.instr)

B.truelist = merge(B1.truelist, B2.truelist)

B.falselist = B2.falselist

AND

$B \rightarrow B1 \ \&\& \ M \ B2$

backpatch(B1.truelist, M.instr)

B.truelist = B2.truelist

B.falselist = merge(B1.falselist, B2.falselist)

NOT

$B \rightarrow ! \ B1$

B.truelist = B1.falselist

B.falselist = B1.truelist

Relational Expression

$B \rightarrow E1 \ \text{relop} \ E2$

emit(if E1 relop E2 goto _)

emit(goto _)

B.truelist = makelist(nextinstr - 2)

B.falselist = makelist(nextinstr - 1)

TRUE

B.truelist = makelist(nextinstr)

emit(goto _)

FALSE

B.falselist = makelist(nextinstr)

emit(goto _)

(ii) Generate TAC using short-circuit evaluation.

Answer

1: if $a < b$ goto _

2: goto _

3: if $c < d$ goto _

4: goto _

5: if $e < f$ goto _

6: goto

7: TRUE

8: FALSE

(iii) Show truelist, falselist, makelist, merge, backpatch, and final code.

Initial Lists

For **(a < b)**

TrueList = {1}

FalseList = {2}

For **(c < d)**

TrueList = {3}

FalseList = {4}

For **(e < f)**

TrueList = {5}

FalseList = {6}

Applying AND

backpatch($\{3\}$, 5)

TrueList = $\{5\}$

FalseList = merge($\{4\}$, $\{6\}$)

= $\{4,6\}$

Applying OR

backpatch($\{2\}$, 3)

TrueList = merge($\{1\}$, $\{5\}$)

= $\{1,5\}$

FalseList = $\{4,6\}$

Final Backpatched Code

1: if $a < b$ goto Ltrue

2: goto L3

L3:

3: if $c < d$ goto L5

4: goto Lfalse

L5:

5: if $e < f$ goto Ltrue

6: goto Lfalse

Ltrue:

...

Lfalse:

...

Important Backpatching Functions

makelist(i)

Creates a new list containing instruction **i**.

merge(p1, p2)

Combines two lists into a single list.

backpatch(list, label)

Fills all pending jump targets in **list** with the specified **label**.